# Unwrap — Confidential Token to ERC-20

> ***Nox Protocol — Developer Guide*** *Arbitrum Sepolia Testnet (chainId: 421614)*

---

## Overview

The **Unwrap** operation converts confidential tokens (e.g. cRLC) back into public ERC-20 tokens (e.g. RLC) at a 1:1 ratio.

This is a **2-step process**: the user first initiates the unwrap (burns cTokens), then finalizes it to receive the public tokens. This two-step design exists because the confidential amount must be decrypted inside the TEE (Trusted Execution Environment) before the public tokens can be released.

**Use case:** A user holds 50 cRLC (encrypted) and wants to convert them back to 50 RLC (public, tradeable).

---

## Architecture

| Component | Path / Address |
|---|---|
| **Hook** | `hooks/use-unwrap.ts` |
| **Modal** | `components/modals/wrap-modal.tsx` (Unwrap tab) |
| **HandleClient** | `@iexec-nox/handle` (client-side only) |
| **cToken ABI** | `lib/confidential-token-abi.ts` |
| **Gas Utility** | `lib/gas.ts` |
| **cRLC Contract** | `0x271f46e78f2fe59817854dabde47729ac4935765` |

---

## Step-by-Step Flow

```
User                HandleClient           cToken (cRLC)
  |                     |                      |
  |-- 1. encryptInput -->|                      |
  |<-- {handle, proof} --|                      |
  |                     |                      |
  |-- 2. unwrap(from, to, handle, proof) ------>|
  |<--- tx receipt + logs -------------------------|
  |                     |                      |
  |   [decode UnwrapRequested event → contractHandle]
  |                     |                      |
  |     [2s cooldown — NoxCompute processing]   |
  |                     |                      |
  |-- 3. finalizeUnwrap(contractHandle, N, 0x) ->|
  |<--- tx receipt ----------------------------|
  |                     |                      |
  | Balance: cRLC -= N   |    RLC += N (public) |
```

### Step 1 — Encrypt Input

The unwrap amount must be encrypted using the Nox HandleClient before being sent to the contract. This ensures the amount remains confidential during the transaction.

```
import { HandleClient } from "@iexec-nox/handle";

const handleClient = new HandleClient({
  chainId: 421614,
  account: walletClient.account,
});

const { handle, handleProof } = await handleClient.encryptInput(
  parsedAmount,    // BigInt — amount in wei
  "uint256",       // Solidity type
  cTokenAddress    // Contract that will use this input
);
```

**Returns:**

| Field | Type | Description |
|---|---|---|
| `handle` | `bytes32` | Encrypted reference to the amount |
| `handleProof` | `bytes` | Zero-knowledge proof of correct encryption |

> ***Warning:*** *HandleClient uses the Web Crypto API — it must only be used client-side (never in SSR/server components).*

---

### Step 2 — Unwrap (Initiate)

Calls `unwrap()` on the confidential token contract. This burns the cTokens and creates an unwrap request.

```
// Called on the cToken contract (e.g. cRLC)
function unwrap(
    address from,
    address to,
    bytes32 encryptedAmount,
    bytes calldata inputProof
) external returns (bytes32);
```

**Parameters:**

| Param | Value | Description |
|---|---|---|
| `from` | User's address | Owner of the cTokens |
| `to` | User's address | Recipient of the public tokens (same as `from`) |
| `encryptedAmount` | `handle` from Step 1 | Encrypted amount reference |
| `inputProof` | `handleProof` from Step 1 | Proof of correct encryption |

**Implementation:**

```
const overrides = await estimateGasOverrides(publicClient);

const unwrapHash = await writeContractAsync({
  address: confidentialAddress,
  abi: confidentialTokenAbi,
  functionName: "unwrap",
  args: [address, address, handle, handleProof],
  ...overrides,
});

const receipt = await waitForTransactionReceipt(config, { hash: unwrapHash });
```

**Critical — Event Decoding:**

After the receipt, you MUST extract the **contract-generated handle** from the `UnwrapRequested` event. This handle is **different** from the encryptInput handle — the contract generates a new one internally via `_burn`.

```
import { decodeEventLog } from "viem";

let contractHandle: `0x${string}` | undefined;

for (const log of receipt.logs) {
  try {
    const decoded = decodeEventLog({
      abi: confidentialTokenAbi,
      data: log.data,
      topics: log.topics,
    });
    if (decoded.eventName === "UnwrapRequested") {
      contractHandle = decoded.args.amount; // bytes32 — the NEW handle
      break;
    }
  } catch {
    // Skip non-matching logs
  }
}
```

**After decoding:** 2-second cooldown for NoxCompute processing.

---

### Step 3 — Finalize Unwrap

Calls `finalizeUnwrap()` to complete the unwrap and receive the public ERC-20 tokens.

```
// Called on the cToken contract (e.g. cRLC)
function finalizeUnwrap(
    bytes32 unwrapAmount,      // Handle from UnwrapRequested event
    uint256 cleartextAmount,   // Amount to release
```

```
    bytes calldata decryptionProof  // Mock for now
) external;
```

**Parameters:**

| Param | Value | Description |
|---|---|---|
| `unwrapAmount` | Handle from `UnwrapRequested` event | NOT the encryptInput handle |
| `cleartextAmount` | Original amount in wei | Same amount as encrypted in Step 1 |
| `decryptionProof` | `"0x00"` | Mock proof (TEE proof system WIP) |

**Implementation:**

```
const finalizeOverrides = await estimateGasOverrides(publicClient);

const finalizeHash = await writeContractAsync({
  address: confidentialAddress,
  abi: confidentialTokenAbi,
  functionName: "finalizeUnwrap",
  args: [contractHandle, parsedAmount, "0x00"],
  ...finalizeOverrides,
});

await waitForTransactionReceipt(config, { hash: finalizeHash });
```

---

## State Machine

```
idle → encrypting → unwrapping → finalizing → confirmed
                        ↘           ↘
                      error       error (retryable)
```

| State | UI | Description |
|---|---|---|
| `idle` | Form visible | User selects token + amount |
| `encrypting` | Spinner | HandleClient encrypting amount |
| `unwrapping` | Step 1 highlighted | Unwrap tx pending |
| `finalizing` | Step 2 highlighted | Finalize tx pending |
| `confirmed` | Success badge + Arbiscan link | Both txs confirmed |
| `error` | Error message + retry | Any step failed |

### Retry Mechanism

If **finalize fails** (Step 3), the cTokens have already been burned (Step 2). The tokens are "in transit" — the user's cRLC is gone but RLC hasn't been received yet.

The hook exposes `retryFinalize()` which re-sends only the finalize transaction using the stored `contractHandle` and `parsedAmount`, without re-encrypting or re-calling unwrap.

```
const { retryFinalize, isFinalizeError } = useUnwrap();

// In error UI:
if (isFinalizeError) {
  <Button onClick={retryFinalize}>Retry Finalize</Button>
}
```

---

## Handle Confusion — Critical Warning

The unwrap flow involves **two different handles**:

| Handle | Source | Used In |
|---|---|---|
| **encryptInput handle** | `handleClient.encryptInput()` | `unwrap()` call (Step 2) |
| **contract handle** | `UnwrapRequested` event | `finalizeUnwrap()` call (Step 3) |

**Never use the encryptInput handle in finalizeUnwrap.** The contract generates a new handle internally when `_burn` is called, and that's the one needed for finalization.

---

## Gas Estimation

**Estimated gas limits:**

- Unwrap (Step 2): ~300,000 gas units
- Finalize (Step 3): ~200,000 gas units

All transactions use the 20% EIP-1559 buffer (see `lib/gas.ts`).

---

## Important Notes

1. **2-step process** — Unwrap requires two separate on-chain transactions (unwrap + finalize). This is by design — the TEE needs to process the burn before releasing public tokens.
2. **Client-side only** — HandleClient uses Web Crypto API. Never import it in server components or API routes.
3. **Mock proof** — `finalizeUnwrap` currently accepts `"0x00"` as the decryption proof. The real TEE proof system is under development.
4. **Tokens in transit** — If unwrap succeeds but finalize fails, cTokens are burned but ERC-20s aren't released. Use `retryFinalize()` to recover.
5. **Handle is ephemeral** — After unwrap, the balance handle changes. Re-read `confidentialBalanceOf` for any subsequent operation.
6. **Event decoding** — Always use `decodeEventLog()` from viem with the full ABI. Never parse logs manually or use heuristic matching.

## Error Handling

| Error | Context | Recovery |
|---|---|---|
| User rejected tx | Any step | Reset and retry |
| Encrypt failed | Step 1 | Check HandleClient init |
| Unwrap reverted | Step 2 | Check cToken balance |
| Finalize reverted | Step 3 | Use `retryFinalize()` |
| Event not found | After Step 2 | Bug — UnwrapRequested should always emit |

## Code Section (Dev Mode)

```
// Step 1: Encrypt the amount
handleClient.encryptInput(amount, "uint256", cRLC_address);

// Step 2: Initiate unwrap (burns cTokens)
cRLC.unwrap(from, to, encryptedHandle, inputProof);

// Step 3: Finalize (releases public tokens)
cRLC.finalizeUnwrap(unwrapHandle, cleartextAmount, decryptionProof);
```